Software Engineering Lab

Lab 20

Zero-Downtime Deployment (ZDD)

Graceful Shutdown, Traffic Draining, & Expand-Contract Database



Lukman / lukman-ss

Deploy tanpa draining memutus request aktif, jobs terhenti, dan schema berubah.

DAMPAK

Interrupt HTTP

Pengguna menerima 502/504 selama pod terminta

DAMPAK

Job loss

Background worker terhenti meninggalkan state parsial

DAMPAK

Schema clash

Versi lama tidak kompatibel dengan kolom baru

In-flight traffic dan background work harus selesai sebelum pod dimatikan.

Pengujian sequential tidak meniru interleaving runtime.

KELEMAHAN

Tidak drain

Server menolak request baru tapi tidak menunggu yang berjalan

KELEMAHAN

Tidak sync

Tidak ada preStop, load balancer masih mengirim traffic

KELEMAHAN

DB lock

Schema berubah tiba-tiba menyebabkan error

Manual review tidak dapat meniru timing concurrent traffic.

Start New Before Stopping Old

READY

New version launches

Load balancer routes to new pods

PRESTOP

Old version drains

Finishes in-flight requests & jobs

TERMINATE

Old version terminates

Database schema contract completed

Model ini memastikan tidak ada downtime dengan menjaga dua versi berjalan simultan.

Alur Zero-Downtime Deployment

INIT

Init

Component start, readiness false

READY

Ready

Health probe 200, traffic starts

PRESTOP

SIGTERM

PreStop delay, drain HTTP & worker

EXIT

Terminate

Pod dies, new version continues

Diagram memperlihatkan urutan health probe, preStop, dan drain.

Server graceful shutdown dengan preStop

```
func (s *Server) Shutdown(ctx context.Context) error {
    s.SetReady(false)
    if s.preStop > 0 {
        select { case <-time.After(s.preStop): case <-ctx.Done(): return ctx.
    }
    return s.srv.Shutdown(ctx)
}
```

CODE PATH

Shutdown menandai tidak ready, menunggu preStop, lalu menutup listener.

Tanpa preStop, load balancer masih mengirim request → 502.

SERI KEGAGALAN

1. SIGTERM

Pod dies instantly

SERI KEGAGALAN

2. In-flight request

Client receives connection reset

SERI KEGAGALAN

3. Worker job

Job terpotong, state inconsistency

Path ini menggambarkan konsekuensi jika tidak ada draining.

Graceful vs Naïve Shutdown

Aspect	Graceful (preStop)	Naïve
Traffic	Drained	Dropped
Jobs	Completed	Aborted
DB Schema	Compatible	Conflict

Tabel menegaskan keuntungan mekanisme graceful.

Tiga langkah mitigasi

MITIGASI

1. Health Probe

Gunakan readiness untuk mengisolasi pod

MITIGASI

2. PreStop delay

Setidaknya latency routing table

MITIGASI

3. Expand-Contract DB

Baca/tulis versi lama & baru simultan

Keputusan berurutan memastikan tidak ada downtime.

Terminal output demo

```
go run ./cmd/demo
... SIGTERM received, graceful shutdown completed
```

CODE PATH

Output menunjukkan server menandai unready, menunggu preStop, lalu selesai.

Batasan bukti

BATAS

Dibuktikan

Graceful shutdown, preStop delay, worker drain

BATAS

Tidak dibuktikan

Distributed load balancer latency >1s, real DB DDL lock contention

Lab terbatas pada in-memory server & worker, bukan pada orchestrator cloud.

PreStop vs Fast termination

PRO & CON

PreStop

Safety, extra latency

PRO & CON

Fast kill

Zero latency but risk 502/504

Pilihan tergantung SLA dan toleransi error.

Worker drain dengan WaitGroup

```
func (w *Worker) Stop(timeout time.Duration) {
    w.enqueueMu.Lock(); w.stopped.Store(true); close(w.jobChan); w.enqueueMu.
    done := make(chan struct{})
    go func(){ w.wg.Wait(); close(done) }()
    select { case <-done: case <-time.After(timeout): w.cancel() }
}
```

CODE PATH

Stop menutup channel, menunggu goroutine, dan memaksa context bila timeout.

Sebelum vs Sesudah graceful shutdown

KONDISI

Sebelum

Request drop, job loss, schema error

KONDISI

Sesudah

All in-flight completed, no DB conflict

Visualisasi perbedaan outcome.

Analisis: Health Probe Timing

KEY FINDINGS

Observation

...

KEY FINDINGS

Implication

...

Deep dive per component.

Analisis: PreStop Duration Impact

KEY FINDINGS

Observation

...

KEY FINDINGS

Implication

...

Deep dive per component.

Analisis: Worker Drain Guarantees

KEY FINDINGS

Observation

...

KEY FINDINGS

Implication

...

Deep dive per component.

Analisis: DB Expand-Contract Compatibility

KEY FINDINGS

Observation

...

KEY FINDINGS

Implication

...

Deep dive per component.

Analisis: End-to-End Latency

KEY FINDINGS

Observation

...

KEY FINDINGS

Implication

...

Deep dive per component.

Checklist produksi Zero-Downtime

- ✓ Implement readiness probe returning 503 until warmup
- ✓ Configure preStop hook >= external routing latency
- ✓ Use Expand-Contract DB schema with backward compatible reads/writes
- ✓ Ensure worker can drain with timeout and context cancellation

Langkah-langkah wajib sebelum release.

Mengabaikan preStop & readiness

PITFALLS

Force kill

Mengakibatkan 502/504

PITFALLS

Schema breaking

Deploy kolom baru tanpa fallback

PITFALLS

Job abort

Background worker terhenti abrupt

Contoh kegagalan yang sering terjadi.

Perintah demo lokal

```
cd labs/20-zero-downtime-deployment
go test -v ./...
go run ./cmd/demo
```

CODE PATH

Perintah menyiapkan lingkungan dan memverifikasi zero-downtime.

Desain untuk koeksistensi versi

PILLARS

Versioned APIs

Maintain backward compatibility

PILLARS

Graceful shutdown

Health probes + preStop

PILLARS

Schema evolution

Expand-Contract pattern

Prinsip ini menjaga uptime selama evolusi sistem.

Zero-Downtime Deployment bukan kegagalan, melainkan strategi.

Penutup menegaskan nilai arsitektural.

Software Engineering Lab 20 - Zero-Downtime Deployment (ZDD)

Repo: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/20-zero-downtime-deployment
Commit:
Author: Lukman / lukman-ss

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/20-zero-downtime-deployment`

Lukman / lukman-ss

Repo: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/20-zero-downtime-deployment
Commit:
Author: Lukman / lukman-ss